UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+ ACCREDITED UNIVERSITY

Worksheet 5

Student Name: Sahil Sharma

UID: 26MCI10257

Branch: MCA (AI & ML)

Section/Group: 26MAM-4B

Semester: 1st

Date of Performance: 16-09-2026

Subject Name: Python (Lab)

Subject Code: 26CAH-606

Experiment 5(a)

Aim:

To write a Python program to generate a simple bar graph using matplotlib. The graph should be properly labeled.

Objectives:

- To understand the process of visualizing discrete, categorical data.
- To learn how to map data arrays to X and Y axes using Python.
- To familiarize with graph formatting techniques, including axis labeling and titling.

Theory:

- A bar graph is a basic 2D visualization tool that I use to represent discrete or categorical data using rectangular bars.
- The physical height of each bar is directly proportional to the numerical value it holds, which makes it very easy to compare different sets of data visually.
- In this experiment, I have used Python's matplotlib library, which is the standard tool for creating static plots and charts.
- I specifically worked with the pyplot submodule (imported as plt) because it provides simple, built-in functions to draw graphs step-by-step.
- To construct the graph, I used the core function plt.bar(). This function requires two main lists from my dataset: one mapped to the X-axis (the categories) and one mapped to the Y-axis (the data values).
- To make the visualization clear and distinct, I customized the appearance using optional parameters like color (for the inside of the bar) and edgecolor (for the borders).

- Finally, to ensure the graph is self-explanatory, I added necessary context using plt.title(), plt.xlabel(), and plt.ylabel(), and then used plt.show() to render the final output on my screen.

Code:

```
import matplotlib.pyplot as plt

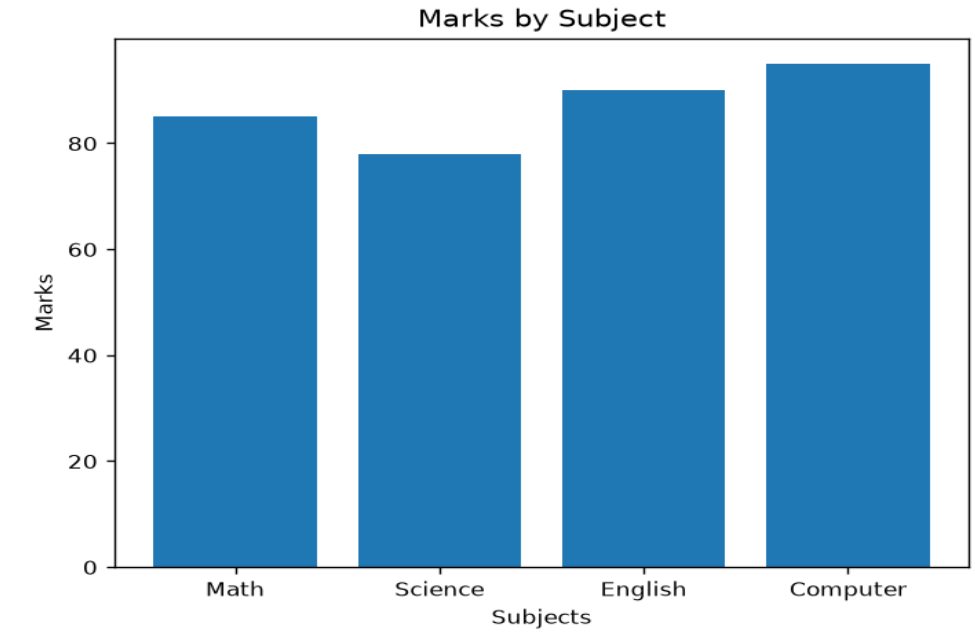
# Data
subjects = ["Math", "Science", "English", "Computer"]
marks = [85, 78, 90, 95]

# Create bar graph
plt.bar(subjects, marks)

# Labels and title
plt.xlabel("Subjects")
plt.ylabel("Marks")
plt.title("Marks by Subject")

# Display graph
plt.show()
```

Output:



Learning Outcomes:

- Successfully import and utilize the matplotlib.pyplot module.
- Construct a vertical bar graph to compare categorical datasets.
- Enhance graph readability by applying standard structural elements like titles and axis labels.

Experiment 5(b)

Aim:

To write a Python program to generate a Pie-chart using matplotlib. The graph should be properly labeled.

Objectives:

- To understand how to represent data as proportions of a whole.
- To utilize the pie() function to map a single array of numerical data into circular sectors.
- To format string outputs within a graph to display automated percentage calculations.

Theory:

- A pie chart is a circular statistical graphic that I use to show how individual categories contribute to a total whole.
- Unlike a bar graph that compares separate magnitudes, a pie chart represents parts of 100%, where the geometric angle and arc length of each slice are strictly proportional to its quantity.
- To build this chart, I again utilized the pyplot submodule from the matplotlib library.
- I used the core function plt.pie(). Interestingly, unlike bar graphs, this function only needs a single list of numerical data as its main argument, because it automatically calculates the degrees of the circle for me behind the scenes.
- To make the chart readable and organized, I passed my category list to the labels parameter so each slice is clearly named.
- I also utilized the autopct parameter (using a string format like %1.1f%%) to instruct Python to automatically calculate and print the exact percentage directly on top of each slice.
- Finally, I added a main title using plt.title() and executed plt.show() to render the circular visualization on my screen.

Code:

```
import matplotlib.pyplot as plt

# Data
labels = ['Python', 'Java', 'C++', 'JavaScript', 'C']
sizes = [35, 25, 20, 15, 5]

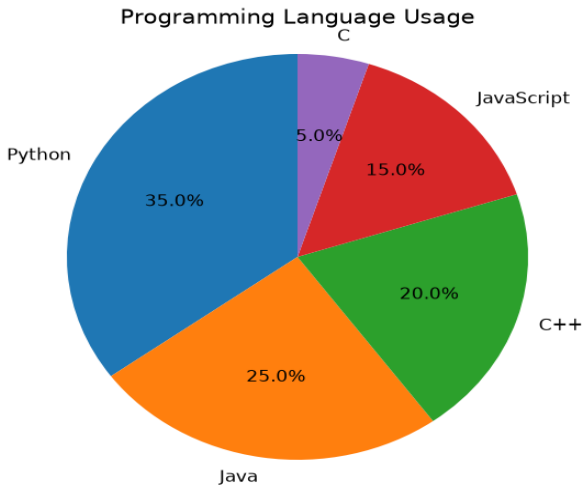
# Create pie chart
plt.pie(
    sizes,
    labels=labels,
    autopct='%1.1f%%',
    startangle=90
)

# Add title
plt.title('Programming Language Usage')

# Ensure the pie chart is circular
plt.axis('equal')

# Display the chart
plt.show()
```

Output:



Learning Outcomes:

- Visualize the composition of a dataset using a proportional pie chart.
- Apply formatting strings (like %1.1f%%) to automatically calculate and display percentages.
- Customize visual aesthetics such as starting angles and sector labels.

Experiment 5(c)

Aim:

To write a Python program to plot the function $y=x2$ using the matplotlib libraries.

Objectives:

- To learn how to generate dense numerical sequences using the numpy library.
- To understand the concept of array vectorization for calculating algebraic outputs.
- To plot continuous mathematical curves using Cartesian coordinates.

Theory:

- To plot a continuous algebraic equation like $y=x2$, I need to generate a dense sequence of Cartesian coordinates (x,y) rather than just a few distinct categorical points.
- For this experiment, I imported the numpy library alongside matplotlib, because standard Python lists are not efficient for applying mathematical formulas to multiple numbers at once.
- I used the np.linspace() function to automatically generate a large array of evenly spaced numbers for my X-axis (the domain), rather than typing a hundred numbers manually.
- To get my Y-axis values, I applied the exponentiation operator (**2) directly to the entire NumPy array. This technique, known as vectorization, instantly computes the corresponding y-value for every x-value simultaneously.
- With my X and Y coordinates prepared, I used the plt.plot() function. This specific function connects all the generated coordinate pairs together to form a smooth, continuous parabolic curve.
- To make the graph look like a proper mathematical worksheet, I enabled the background grid using the plt.grid(True) command.
- I finished the experiment by applying descriptive axis labels and a title to clearly define the function, and then rendered the final curve using plt.show().

Code:

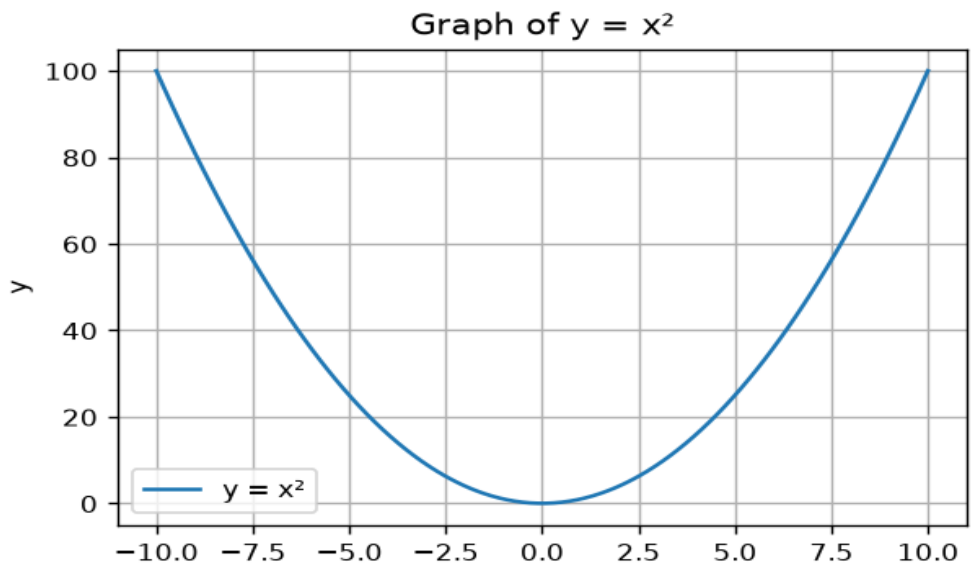
```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(-10, 10, 100)

y = x ** 2

plt.plot(x, y, label='y = x²')
plt.xlabel('x')
plt.ylabel('y')
plt.title('Graph of y = x²')
plt.grid(True)
plt.legend()
plt.show()
```

Output:



Learning Outcomes:

- Integrate numpy with matplotlib for mathematical plotting.
- Generate evenly spaced numerical domains using linspace.
- Translate standard algebraic equations into vectorized Python code to generate smooth curve visualizations.